chipbag

Requirement verification traceability and gap analysis automated

Maguire Papay

Problem

- Specs are largely prose, test planning translates them to test cases in a lossy projection, discarding all ties between a spec and the test case meant to verify it
- More mature teams with structured requirements keep links between requirements and test cases up to date manually

Current Tooling

- vManager, Verdi, and Questa VIQ trace plan → coverage → results well.
- Jama, DOORS, and Jira can handle requirements → plan and integrate with above

So What?

- DV leads spend several hours to days per IP block to maintain requirement to verification links and pass test plan review and sign off
- Teams that don’t do this well experience drift between their spec and implementation

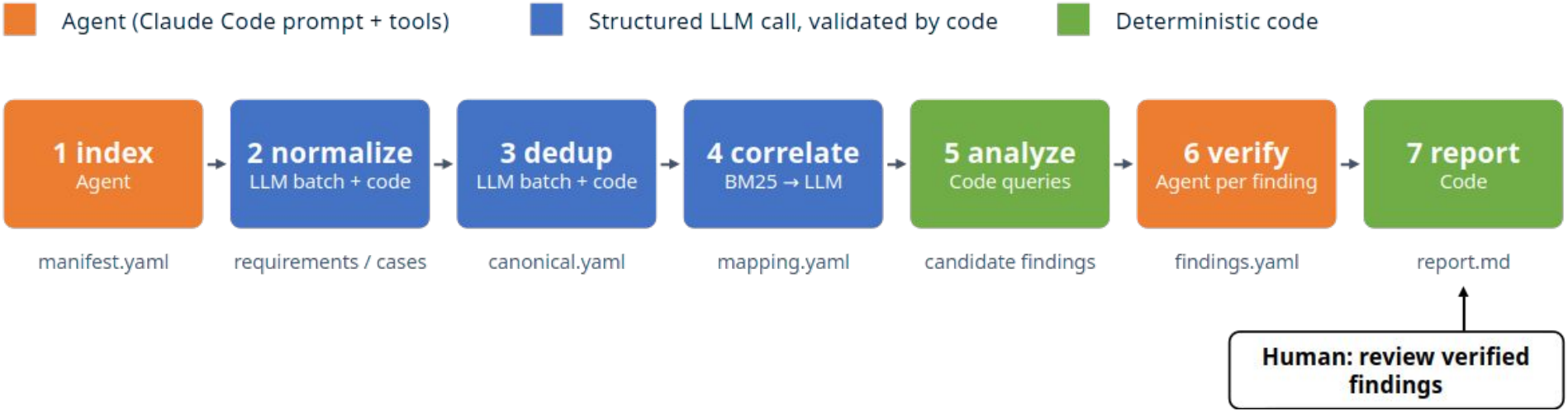
Done Right

- Specs should come with structured requirement trees
- All requirements should trace to the test plan that verifies the requirement
- Requirements gaps and drifts should be continuously evaluated and rectified



chipbag is an agentic pipeline to keep these traces fresh and analyze gaps

Pipeline



Success is the number of reports that a human attests to and takes action on coupled with time/cost saving to generate these findings.

Usage

```
→  chipbag git:(main) ✗ uv run chipbag run --name caliptra-ss-mci-mailbox --pdf
Workspace work/caliptra-ss-mci-mailbox  ·  MCI mailbox  ·  targets/caliptra-ss @ 69a8afa1ee  ·  cost cap $50.00
Scope from work/caliptra-ss-mci-mailbox/scope.yaml: 82 section patterns, 0 chunk patterns
[1/7] requirements  522 requirements from 82 spec sections  ($0.00, 2.5s)
[2/7] cases         196 verification cases from 51 chunks; 244 checks  ($0.00, 1.0s)
[3/7] dedup         487 requirements (363 need a test); 39 tests, 46 plan items  ($0.00, 3.8s)
[4/7] correlate     185 of 363 requirements matched to a test  ($0.00, 5.6s)
[5/7] analyze       94 candidate findings: 1) 19, 2) 4, 3) 16, 4) 2, 5) 53  ($0.00, 5.0s)
[6/7] verify        71 verified, 22 withdrawn, 1 inconclusive  ($0.00, 2.0s)
[7/7] report        written to work/caliptra-ss-mci-mailbox/reports/2026-11-10T214710Z.md, work/caliptra-ss-mci-mailbox/reports/2026-11-10T214710Z.pdf  ($0.00, 8.0s)

Done (complete): $0.00, 28s
Report: work/caliptra-ss-mci-mailbox/reports/2026-11-10T214710Z.md
Report: work/caliptra-ss-mci-mailbox/reports/2026-11-10T214710Z.pdf
→  chipbag git:(main) ✗
```

*Cost is 0 as this is a cached run

Results

Block	Unique requirements	Candidate findings	Verified gaps	Run cost	Run time
Caliptra MCI mailbox	487	94	71	$13.50	18 min
OpenTitan HMAC	411	78	43	$9.63	15 min
OpenTitan AON timer	143	40	22	$7.29	18 min

Results - MCI Mailbox

Stage	Result
Index	44 sources found: 4 spec, 40 verification
Requirements	522 requirements from 82 spec sections; 99% of spec text cited
Cases	196 verification cases, 244 checks; 0 citation failures
Dedup	487 unique requirements (363 need a test); 39 tests, 46 plan items
Correlate	10,610 requirement–test pairs judged; 185 requirements matched to a test, 167 to none
Analyze	94 candidate findings
Verify	71 verified, 22 withdrawn, 2 reclassified, 1 inconclusive

Honest Results - MCI Mailbox

Category	Checked	Real	False positive
1. Spec too ambiguous to test	7	7	0
2. Spec contradicts itself	1	0	1
3. Test contradicts its own description	12	9	3
Total	20	16	4 (20%)

Example Gap Report

2. **Requirement** `cr_bb5623ddc32e`: Only the trusted AXI users, the default user 0xFFFF_FFFF, and MCU can access or obtain a lock on the mailbox. Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Limited Trusted AXI users

   - `docs/CaliptraSSHardwareSpecification.md:1297`: `These users, a default user 0xFFFF_FFFF, and MCU are the only AXI users that can access or obtain a lock on the mailbox.`

   Tested by `smoke_test_mcu_mbox_invalid_axi`**:** A user outside the allowed set gets AXI errors on all CSR and SRAM accesses. **Not tested:** Does not check lock acquisition by such a user, nor that the default user 0xFFFF_FFFF or MCU are allowed.

   Verified: No test checks lock denial for an untrusted user, or access by the default user.

Example Gap Report Pt 2

1.1 Post-MCI-reset SRAM_OWNER encoding: Root (2) or lock holder (1)?

`f_7dbfd12edb46` · found during verification · severity high

Evidence:

- `docs/CaliptraSSHardwareSpecification.md:1395`: `| MCI reset | Root, as the reset lock holder |` — The event table names Root as the owner after MCI reset, but qualifies it as the "reset lock holder".
- `docs/CaliptraSSHardwareSpecification.md:1444-1445`: `| 1 | Requester/lock holder |` — Encoding 1 is labelled "Requester/lock holder", so a lock holder owner can legitimately read back as 1. This supports the test's expectation.
- `src/integration/test_suites/smoke_test_mcu_mbox_csrs_warm_rst/smoke_test_mcu_mbox_csrs_warm_rst.c:113-114`: `if(mcu_mbox_read_sram_owner(mbox_num) != MCU_MBOX_SRAM_OWNER_REQUESTER) {` — The test expects encoding 1 and its message says "reset lock holder", so its author read "lock holder" as the encoding-1 owner.

Verified: The spec's event table says the owner after MCI reset is "Root, as the reset lock holder". The SRAM_OWNER encoding table separates 1 (Requester/lock holder) from 2 (Root). The test expects 1. A spec change that states the exact SRAM_OWNER value after reset would resolve it.

Searched:

- SRAM_OWNER encoding in the spec and library header in `docs/CaliptraSSHardwareSpecification.md, caliptra_ss_lib.h`: 1 = Requester/lock holder, 2 = Root. The spec does not say which encoding applies after reset.

Implementation Considerations

- Both target repos use different methods for maintaining their specs and test plans, need an intelligent layer to bridge the gap
- Breaking into an LLM pipeline gives more consistent results and lower costs than direct agentic action
- Directed pipelines allow us to utilize Sonnet for all model calls, keeping costs low
- Treat all information as plausible until verification agent runs
- Produce the smallest possible surface area for a human reviewer to check and confirm

What Went Wrong

- Files missing from an index cannot be detected, and influence precision and recall in future stages
- Run-to-run variance exists depending on AI choices and early stages impact later ones heavily
- Verifier agent still produces false positives in its report

AI Usage

- AI is uniquely fit for this task as it requires many repetitive intelligent tasks
- Early-stage mistakes mostly cost recall; verify filters most of them, not all (3 of the 4 false positives started in correlate)
- Verify stage dispatches individual agents to confirm the gathered results, mistakes here get reported to the user as false findings
- AI findings cannot be treated as a complete list of all possible gaps

Future Work

- Integrate with real requirement handling software
- Implement triggers for the pipeline to run (CI and hashing)
- Additional functionality for greenfield projects doing requirement > test plan decomposition
- At scale, build chipbag in a way to reuse as much computation for incremental invocations